

HTTP API Reference

Routes, parameters, and response shapes

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

The Flask application exposes both HTML pages and a comprehensive JSON API. This document enumerates every route, lists their parameters with default values, and gives the schema of the JSON responses, including new features in v0.3.0.

1. Routes

Method	Path	Description
GET	/	Landing page with search form, examples, feature grid.
GET	/about	Project overview, API summary, links.
GET	/search	HTML results page with facets and bookmarks.
GET	/api/search	Ranked results as JSON with filters/sorting.
GET	/api/semantic-search	Semantic search using TF-IDF vectors.
POST	/api/batch-search	Search multiple queries at once (max 20).
GET	/api/autocomplete	Query autocomplete suggestions.
GET	/api/export	Export results as JSON or CSV.
GET	/api/analytics	Search analytics statistics.
GET	/api/top-queries	Top queries by frequency.
GET	/api/related-queries	Related queries for a given query.
POST	/api/documents	Add a new document to the index.
DELETE	/api/documents/<id>	Remove a document from the index.
GET	/api/stats	Corpus statistics as JSON.
GET	/metrics	Prometheus-compatible metrics.
GET	/api/metrics	JSON metrics (P95, P99, hit rate).
GET	/healthz	Liveness probe.
GET	/readyz	Readiness check (for Kubernetes).

2. GET /api/search

Run a query and return ranked results as JSON. Supports facets, sorting, and all query syntax options.

2.1 Query parameters

Name	Type	Default	Description
q	string	required	Raw query text.
model	string	bm25	One of bm25, tfidf, semantic.
k	int	10	Number of results, clamped to 1..50.
domain	string	null	Filter by domain.
source	string	null	Filter by source.
sort	string	relevance	Sort by relevance/title/score_asc.

2.2 Example

```
curl "http://127.0.0.1:5000/api/search?q=machine+learning&model=bm25&k=3"
```

2.3 Response shape

```
{
  "query": "machine learning",
  "model": "bm25",
  "top_k": 3,
  "results": [
    {
      "doc_id": 1,
      "title": "machine learning",
      "snippet": "<mark>Machine</mark> <mark>learning</mark> ...",
      "score": 3.59904,
      "source": "machine_learning.txt",
    }
  ],
  "suggestion": null,
  "facets": {...}
}
```

If the query returns no results, the suggestion field contains a spell-corrected variant produced by the Damerau-Levenshtein checker; otherwise it is null.

3. GET /api/semantic-search

Perform semantic search using TF-IDF vector embeddings. Returns results ranked by cosine similarity.

```
curl "http://127.0.0.1:5000/api/semantic-search?q=machine+learning&k=5"

{
  "query": "machine learning",
  "model": "semantic",
  "top_k": 5,
  "results": [...],
  "time_taken": 0.123
}
```

4. POST /api/batch-search

Search multiple queries at once (max 20 queries per batch).

```
curl -X POST "http://127.0.0.1:5000/api/batch-search" \
-H "Content-Type: application/json" \
-d '{"queries": ["ml", "nlp"], "top_k": 5}'

{
  "results": {
    "ml": [...],
    "nlp": [...]
  },
  "query_count": 2,
}
```

5. GET /api/export

Export search results in JSON or CSV format.

```
curl "http://127.0.0.1:5000/api/export?q=ml&format=csv"
```

6. GET /api/autocomplete

Get autocomplete suggestions for a prefix as you type.

```
curl "http://127.0.0.1:5000/api/autocomplete?prefix=ma"

{
  "prefix": "ma",
  "suggestions": ["machin", "main", "make"]
}
```

7. Analytics Endpoints

```
curl "http://127.0.0.1:5000/api/analytics"
curl "http://127.0.0.1:5000/api/top-queries"
curl "http://127.0.0.1:5000/api/related-queries?q=ml"
```

8. Document Management

Add or remove documents via API endpoints.

```
# Add document (POST)
curl -X POST "http://127.0.0.1:5000/api/documents" \
-H "Content-Type: application/json" \
-d '{"title": "Doc", "text": "content"}'

# Remove document (DELETE)
curl -X DELETE "http://127.0.0.1:5000/api/documents/1"
```

9. Monitoring Endpoints

```
# Prometheus metrics
curl "http://127.0.0.1:5000/metrics"

# JSON metrics
curl "http://127.0.0.1:5000/api/metrics"

# Health check
curl "http://127.0.0.1:5000/healthz"

# Readiness check
curl "http://127.0.0.1:5000/readyz"
```

10. Error Handling

Condition	Response
Unknown model	400 with {"error": "unknown model 'X'"}
Missing q	200 with empty results
Out-of-range k	Silently clamped to [1, 50]

11. Calling the API

11.1 Python

```
import requests

resp = requests.get(
    "http://127.0.0.1:5000/api/search",
    params={"q": "neural networks", "model": "tfidf", "k": 5},
    timeout=5,
)
for hit in resp.json()["results"]:
    print(f"{hit[\"score\"]}:6.3f {hit[\"title\"]}")
```

11.2 JavaScript

```
const res = await fetch(
    '/api/search?' + new URLSearchParams({q: 'neural', model: 'bm25'})
);
const data = await res.json();
console.log(data.results);
```